

Recipe RAG Assistant

Problem Statement and Project Requirements

Build a Python Streamlit web app called **Recipe RAG Assistant** — a Document Q&A RAG agent that acts as an intelligent recipe generator.

TECH STACK (Mandatory)

- **Frontend/App:** Streamlit
- **LLM:** Groq API, model `openai/gpt-oss-120b`
- **Embeddings + Vector Store:** sentence-transformers (all-MiniLM-L6-v2) + FAISS (local, in-memory)
- **Language:** Python 3.11

FEATURES TO IMPLEMENT

1. Document Ingestion

- Let the user upload recipe files (PDF or .txt) via `st.file_uploader`, or load a preloaded sample folder of ~20 recipes (title, ingredients, steps, cuisine, dietary tags, prep time).
- Chunk the documents and embed them, storing vectors in FAISS in `session_state` so it persists during the session.

2. Conversational Q&A (RAG)

- A chat interface (`st.chat_input` / `st.chat_message`) where the user asks natural questions like "How can I make a sugar-free chocolate cake?"
- Retrieve top-k relevant chunks from FAISS, build a context-augmented prompt, and call the Groq API (model="openai/gpt-oss-120b") to generate the answer.
- Show retrieved source recipe names for transparency.

3. Personalization Layer

- Sidebar inputs: dietary restrictions (vegan, gluten-free, diabetic, etc.), available ingredients (text box), max cooking time, cuisine preference.
- When generating an answer, inject these preferences into the LLM prompt so recipes/substitutions are adapted accordingly.

4. Output Formatting

- Present the final answer as: step-by-step numbered instructions, a substitutions box, an estimated nutrition facts table, and an auto-generated shopping list (bullet list of ingredients not already marked available).

CONFIG

- Load `GROQ_API_KEY` from environment variable / `.env` file (use `python-dotenv`).

- Add a requirements.txt with streamlit, groq, faiss-cpu, sentence-transformers, pypdf, python-dotenv.
- Add a clean README.md explaining setup and how to run (streamlit run app.py).

PROJECT STRUCTURE

- app.py (main Streamlit app)
- rag_utils.py (chunking, embedding, FAISS index, retrieval functions)
- llm_utils.py (Groq API call wrapper, prompt templates)
- sample_data/ (a few example recipe .txt files)
- requirements.txt
- README.md

UI AND ERROR HANDLING

Keep the UI clean and simple: a sidebar for preferences/upload, and a main chat panel for the Q&A. Add basic error handling for missing API key and empty vector store.